

# Sistemas distribuidos

## Introducción

Un sistema distribuido se define como una colección de computadores autónomos conectados por una red, y con el software distribuido adecuado para que el sistema sea visto por los usuarios como una única entidad capaz de proporcionar facilidades de computación. [ Colouris 1994 ]

El desarrollo de los sistemas distribuidos vino de la mano de las redes locales de alta velocidad a principios de 1970. Mas recientemente, la disponibilidad de computadoras personales de altas prestaciones, estaciones de trabajo y ordenadores servidores ha resultado en un mayor desplazamiento hacia los sistemas distribuidos en detrimento de los ordenadores centralizados multiusuario. Esta tendencia se ha acelerado por el desarrollo de software para sistemas distribuidos, diseñado para soportar el desarrollo de aplicaciones distribuidas. Este software permite a los ordenadores coordinar sus actividades y compartir los recursos del sistema - hardware, software y datos.

Los sistemas distribuidos se implementan en diversas plataformas hardware, desde unas pocas estaciones de trabajo conectadas por una red de área local, hasta Internet, una colección de redes de área local y de área extensa interconectados, que enlazan millones de ordenadores.

Las aplicaciones de los sistemas distribuidos varían desde la provisión de capacidad de computo a grupos de usuarios, hasta sistemas bancarios, comunicaciones multimedia y abarcan prácticamente todas las aplicaciones comerciales y técnicas de los ordenadores. Los requisitos de dichas aplicaciones incluyen un alto nivel de fiabilidad, seguridad contra interferencias externas y privacidad de la información que el sistema mantiene. Se deben proveer accesos concurrentes a bases de datos por parte de muchos usuarios, garantizar tiempos de respuesta, proveer puntos de acceso al servicio que están distribuidos geográficamente, potencial para el crecimiento del sistema para acomodar la expansión del negocio y un marco para la integración de sistemas usados por diferentes compañías y organizaciones de usuarios.

## Características clave de los sistemas distribuidos

[Colouris 1994] establece que son seis las características principales responsables de la utilidad de los sistemas distribuidos. Se trata de compartición de recursos, apertura (openness), concurrencia, escalabilidad, tolerancia a fallos y transparencia. En las siguientes líneas trataremos de abordar cada una de ellas.

### **Compartición de Recursos**

El término 'recurso' es bastante abstracto, pero es el que mejor caracteriza el abanico de entidades que pueden compartirse en un sistema distribuido. El abanico se extiende desde componentes hardware como discos e impresoras hasta elementos software como archivos, ventanas, bases de datos y otros objetos de datos.

La idea de compartición de recursos no es nueva ni aparece en el marco de los sistemas distribuidos. Los sistemas multiusuario clásicos desde siempre han provisto compartición de recursos entre sus usuarios. Sin embargo, los recursos de una computadora multiusuario se comparten de manera

natural entre todos sus usuarios. Por el contrario, los usuarios de estaciones de trabajo monousuario o computadoras personales dentro de un sistema distribuido no obtienen automáticamente los beneficios de la compartición de recursos.

Los recursos en un sistema distribuido están físicamente encapsulados en una de las computadoras y sólo pueden ser accedidos por otras computadoras mediante las comunicaciones (la red). Para que la compartición de recursos sea efectiva, ésta debe ser manejada por un programa que ofrezca un interfaz de comunicación permitiendo que el recurso sea accedido, manipulado y actualizado de una manera fiable y consistente. Surge el término genérico de **gestor de recursos**.

Un gestor de recursos es un modulo software que maneja un conjunto de recursos de un tipo en particular. Cada tipo de recurso requiere algunas políticas y métodos específicos junto con requisitos comunes para todos ellos. Éstos incluyen la provisión de un esquema de nombres para cada clase de recurso, permitir que los recursos individuales sean accedidos desde cualquier localización; la traslación de nombre de recurso a direcciones de comunicación y la coordinación de los accesos concurrentes que cambian el estado de los recursos compartidos para mantener la consistencia.

Un sistema distribuido puede verse de manera abstracta como un conjunto de gestores de recursos y un conjunto de programas que usan los recursos. Los usuarios de los recursos se comunican con los gestores de los recursos para acceder a los recursos compartidos del sistema. Esta perspectiva nos lleva a dos modelos de sistemas distribuidos: el modelo **cliente-servidor** y el modelo **basado en objetos**.

## **Apertura (openness)**

Un sistema informático es abierto si el sistema puede ser extendido de diversas maneras. Un sistema puede ser abierto o cerrado con respecto a extensiones hardware (añadir periféricos, memoria o interfaces de comunicación, etc... ) o con respecto a las extensiones software (añadir características al sistema operativo, protocolos de comunicación y servicios de compartición de recursos, etc... ). La apertura de los sistemas distribuidos se determina primariamente por el grado hacia el que nuevos servicios de compartición de recursos se pueden añadir sin perjudicar ni duplicar a los ya existentes.

Básicamente los sistemas distribuidos cumplen una serie de características:

1. Los interfaces software clave del sistema están claramente especificados y se ponen a disposición de los desarrolladores. En una palabra, los interfaces se hacen públicos.
2. Los sistemas distribuidos abiertos se basan en la provisión de un mecanismo uniforme de comunicación entre procesos e interfaces publicados para acceder a recursos compartidos.
3. Los sistemas distribuidos abiertos pueden construirse a partir de hardware y software heterogéneo, posiblemente proveniente de vendedores diferentes. Pero la conformidad de cada componente con el estándar publicado debe ser cuidadosamente comprobada y certificada si se quiere evitar tener problemas de integración.

## **Concurrencia**

Cuando existen varios procesos en una única máquina decimos que se están ejecutando concurrentemente. Si el ordenador está equipado con un único procesador central, la concurrencia tiene lugar entrelazando la ejecución de los distintos procesos. Si la computadora tiene N procesadores, entonces se pueden estar ejecutando estrictamente a la vez hasta N procesos.

En los sistemas distribuidos hay muchas maquinas, cada una con uno o mas procesadores centrales. Es decir, si hay  $M$  ordenadores en un sistema distribuido con un procesador central cada una entonces hasta  $M$  procesos estar ejecutándose en paralelo.

En un sistema distribuido que esta basado en el modelo de compartición de recursos, la posibilidad de ejecución paralela ocurre por dos razones:

1. Muchos usuarios interactuan simultáneamente con programas de aplicación.
2. Muchos procesos servidores se ejecutan concurrentemente, cada uno respondiendo a diferentes peticiones de los procesos clientes.

El caso (1) es menos conflictivo, ya que normalmente las aplicaciones de interacción se ejecutan aisladamente en la estación de trabajo del usuario y no entran en conflicto con las aplicaciones ejecutadas en las estaciones de trabajo de otros usuarios.

El caso (2) surge debido a la existencia de uno o mas procesos servidores para cada tipo de recurso. Estos procesos se ejecutan en distintas maquinas, de manera que se están ejecutando en paralelo diversos servidores, junto con diversos programas de aplicación. Las peticiones para acceder a los recursos de un servidor dado pueden ser encoladas en el servidor y ser procesadas secuencialmente o bien pueden ser procesadas varias concurrentemente por múltiples instancias del proceso gestor de recursos. Cuando esto ocurre los procesos servidores deben sincronizar sus acciones para asegurarse de que no existen conflictos. La sincronización debe ser cuidadosamente planeada para asegurar que no se pierden los beneficios de la concurrencia.

## **Escalabilidad**

Los sistemas distribuidos operan de manera efectiva y eficiente a muchas escalas diferentes. La escala más pequeña consiste en dos estaciones de trabajo y un servidor de archivos, mientras que un sistema distribuido construido alrededor de una red de área local simple podría contener varios cientos de estaciones de trabajo, varios servidores de archivos, servidores de impresión y otros servidores de propósito específico. A menudo se conectan varias redes de área local para formar **internetworks**, y éstas podrían contener muchos miles de ordenadores que forman un único sistema distribuido, permitiendo que los recursos sean compartidos entre todos ellos.

Tanto el software de sistema como el de aplicación no deberían cambiar cuando la escala del sistema se incrementa. La necesidad de escalabilidad no es solo un problema de prestaciones de red o de hardware, sino que esta íntimamente ligada con todos los aspectos del diseño de los sistemas distribuidos. El diseño del sistema debe reconocer explícitamente la necesidad de escalabilidad o de lo contrario aparecerán serias limitaciones.

La demanda de escalabilidad en los sistemas distribuidos ha conducido a una filosofía de diseño en que cualquier recurso simple -hardware o software- puede extenderse para proporcionar servicio a tantos usuarios como se quiera. Esto es, si la demanda de un recurso crece, debería ser posible extender el sistema para darla servicio,. Por ejemplo, la frecuencia con la que se accede a los archivos crece cuando se incrementa el numero de usuarios y estaciones de trabajo en un sistema distribuido. Entonces, debe ser posible añadir ordenadores servidores para evitar el cuello de botella que se produciría si un solo servidor de archivos tuviera que manejar todas las peticiones de acceso a los archivos. En este caso el sistema deberá estar diseñado de manera que permita trabajar con archivos replicados en distintos servidores, con las consideraciones de consistencias que ello conlleva.

Cuando el tamaño y complejidad de las redes de ordenadores crece, es un objetivo primordial

diseñar software de sistema distribuido que seguirá siendo eficiente y útil con esas nuevas configuraciones de la red. Resumiendo, el trabajo necesario para procesar una petición simple para acceder a un recurso compartido debería ser prácticamente independiente del tamaño de la red. Las técnicas necesarias para conseguir estos objetivos incluyen el uso de datos replicados, la técnica asociada de caching, y el uso de múltiples servidores para manejar ciertas tareas, aprovechando la concurrencia para permitir una mayor productividad. Una explicación completa de estas técnicas puede encontrarse en [ Colouris 1994 ].

## **Tolerancia a Fallos**

Los sistemas informáticos a veces fallan. Cuando se producen fallos en el software o en el hardware, los programas podrían producir resultados incorrectos o podrían pararse antes de terminar la computación que estaban realizando. El diseño de sistemas tolerantes a fallos se basa en dos cuestiones, complementarias entre sí: Redundancia hardware (uso de componentes redundantes) y recuperación del software (diseño de programas que sean capaces de recuperarse de los fallos).

En los sistemas distribuidos la redundancia puede plantearse en un grano mas fino que el hardware, pueden replicarse los servidores individuales que son esenciales para la operación continuada de aplicaciones críticas.

La recuperación del software tiene relación con el diseño de software que sea capaz de recuperar (roll-back) el estado de los datos permanentes antes de que se produjera el fallo.

Los sistemas distribuidos también proveen un alto grado de disponibilidad en la vertiente de fallos hardware. La disponibilidad de un sistema es una medida de la proporción de tiempo que esta disponible para su uso. Un fallo simple en una maquina multiusuario resulta en la no disponibilidad del sistema para todos los usuarios. Cuando uno de los componentes de un sistema distribuidos falla, solo se ve afectado el trabajo que estaba realizando el componente averiado. Un usuario podría desplazarse a otra estación de trabajo; un proceso servidor podría ejecutarse en otra maquina.

## **Transparencia**

La transparencia se define como la ocultación al usuario y al programador de aplicaciones de la separación de los componentes de un sistema distribuido, de manera que el sistema se percibe como un todo, en vez de una colección de componentes independientes. La transparencia ejerce una gran influencia en el diseño del software de sistema.

El manual de referencia RM-ODP [ISO 1996a] identifica ocho formas de transparencia. Estas proveen un resumen útil de la motivación y metas de los sistemas distribuidos. Las transparencias definidas son:

- Transparencia de Acceso : Permite el acceso a los objetos de información remotos de la misma forma que a los objetos de información locales.
- Transparencia de Localización: Permite el acceso a los objetos de información sin conocimiento de su localización
- Transparencia de Concurrencia: Permite que varios procesos operen concurrentemente utilizando objetos de información compartidos y de forma que no exista interferencia entre ellos.
- Transparencia de Replicación: Permite utilizar múltiples instancias de los objetos de información para incrementar la fiabilidad y las prestaciones sin que los usuarios o los programas de aplicación tengan por que conocer la existencia de las replicas.

- Transparencia de Fallos: Permite a los usuarios y programas de aplicación completar sus tareas a pesar de la ocurrencia de fallos en el hardware o en el software.
- Transparencia de Migración: Permite el movimiento de objetos de información dentro de un sistema sin afectar a los usuarios o a los programas de aplicación.
- Transparencia de Prestaciones. Permite que el sistema sea reconfigurado para mejorar las prestaciones mientras la carga varía.
- Transparencia de Escalado: Permite la expansión del sistema y de las aplicaciones sin cambiar la estructura del sistema o los algoritmos de la aplicación.

Las dos mas importantes son las transparencias de acceso y de localización; su presencia o ausencia afecta fuertemente a la utilización de los recursos distribuidos. A menudo se las denomina a ambas transparencias de red. La transparencia de red provee un grado similar de anonimato en los recursos al que se encuentra en los sistemas centralizados.

## El Modelo Cliente Servidor

El modelo cliente-servidor de un sistema distribuido es el modelo más conocido y más ampliamente adoptado en la actualidad. Hay un conjunto de procesos servidores, cada uno actuando como un gestor de recursos para una colección de recursos de un tipo, y una colección de procesos clientes, cada uno llevando a cabo una tarea que requiere acceso a algunos recursos hardware y software compartidos. Los gestores de recursos a su vez podrían necesitar acceder a recursos compartidos manejados por otros procesos, así que algunos procesos son ambos clientes y servidores. En el modelo, cliente-servidor, todos los recursos compartidos son mantenidos y manejados por los procesos servidores. Los procesos clientes realizan peticiones a los servidores cuando necesitan acceder a algún recurso. Si la petición es válida, entonces el servidor lleva a cabo la acción requerida y envía una respuesta al proceso cliente.

El termino proceso se usa aquí en el sentido clásico de los sistemas operativos. Un proceso es un programa en ejecución. Consiste en un entorno de ejecución con al menos un thread de control.

El modelo cliente-servidor nos da un enfoque efectivo y de propósito general para la compartición de información y de recursos en los sistemas distribuidos. El modelo puede ser implementado en una gran variedad de entornos software y hardware. Las computadoras que ejecuten los programas clientes y servidores pueden ser de muchos tipos y no existe la necesidad de distinguir entre ellas; los procesos cliente y servidor pueden incluso residir en la misma maquina.

En esta visión simple del modelo cliente-servidor, cada proceso servidor podría ser visto como un proveedor centralizado de los recursos que maneja. La provisión de recursos centralizada no es deseable en los sistemas distribuidos. Es por esta razón por lo que se hace una distinción entre los servicios proporcionados a los clientes y los servidores encargados de proveer dichos servicios. Se considera un servicio como una entidad abstracta que puede ser provista por varios procesos servidores ejecutándose en computadoras separadas y cooperando vía red.

El modelo cliente-servidor se ha extendido y utilizado en los sistemas actuales con servicios manejando muchos diferentes tipos de recursos compartidos - correo electrónico y mensajes de noticias, archivos, sincronización de relojes, almacenamiento en disco, impresoras, comunicaciones de área extensa, e incluso las interfaces gráficas de usuario. Pero no es posible que todos los recursos que existen en un sistema distribuido sean manejados y compartidos de esta manera; algunos tipos de recursos deben permanecer locales a cada computadora de cara a una mayor eficiencia - RAM, procesador, interfaz de red local -. Estos recursos clave son manejados

separadamente por un sistema operativo en cada maquina; solo podrían ser compartidos entre procesos localizados en el mismo ordenador.

Aunque el modelo cliente-servidor no satisface todos los requisitos necesarios para todas las aplicaciones distribuidas, es adecuado para muchas de las aplicaciones actuales y provee una base efectiva para los sistemas operativos distribuidos de propósito general.

## Middleware

El término middleware se discute en [Lewandosky 1998]. El software distribuido requerido para facilitar las interacciones cliente-servidor se denomina middleware. El acceso transparente a servicios y recursos no locales distribuidos a través de una red se provee a través del middleware, que sirve como marco para la comunicaciones entre las porciones cliente y servidor de un sistema.

El middleware define: el API que usan los clientes para pedir un servicio a un servidor, la transmisión física de la petición vía red, y la devolución de resultados desde el servidor al cliente. Ejemplos de middleware estándar para dominios específicos incluyen: ODBC, para bases de datos, Lotus para groupware, HTTP y SSL para Internet y CORBA, DCOM y JAVA RMI para objetos distribuidos.

El middleware fundamental o genérico es la base de los sistemas cliente-servidor. Los servicios de autenticación en red, llamadas a procedimiento remoto, sistemas de archivos distribuidos y servicios de tiempo en red se consideran parte del middleware genérico. Este tipo de middleware empieza a ser parte estándar de los sistemas operativos modernos como Windows NT. En sistemas donde no se disponga deberá recurrirse a middleware del tipo OSD DCE (Distributed Computing Environment) [OSF 1994]. El middleware específico para un dominio complementa al middleware genérico de cara a aplicaciones mucho mas específicas.

El protocolo de comunicaciones mas usado por el middleware, tanto genérico como específico, es TCP/IP. Esto se debe a su amplia difusión en todos los sistemas operativos del mercado y en especial en los ordenadores personales.

### **Middleware para sistemas cliente-servidor: RPC**

Un servicio proporcionado por un servidor no es más que un conjunto de operaciones disponibles para los clientes. El acceso al servicio se realiza mediante un protocolo de peticiones respuesta con llamadas bloqueantes. Ejemplo: Un servicio de archivos. El servidor mantiene como recurso compartido los archivos. Sobre el recurso compartido se pueden realizar diversas operaciones: Crear, Abrir, Leer, etc.

Los mecanismos RPC persiguen que los clientes se abstraigan e invoquen procedimientos remotos (operaciones) para obtener servicios. Así, el procedimiento llamado se ejecuta en otro proceso de otra maquina (servidor). El objetivo de RPC es mantener la semántica de la llamada a procedimiento normal en un entorno de implementación totalmente distinto. La ventaja esta en que el desarrollador se preocupa de los interfaces que soporta el servidor. Para especificar dichos interfaces se dispone de un IDL (lenguaje de definición de interfaces).

Los sistemas RPC disponen de mecanismos de RPC integrados en un lenguaje de programación particular que incluye además una notación para definir interfaces entre clientes y servidores (IDL específico). Un IDL permite definir el nombre de las operaciones soportadas por el servidor y sus parámetros (tipo y dirección). También se deben proveer mecanismos para manejo de excepciones,

garantizar la ejecución de las operaciones, así como la detección de fallos. Todo ello de la forma mas transparente posible.

El software (middleware) que soporta RPC tiene tres tareas fundamentales:

1. Procesamiento relacionado con los interfaces: Integrar RPC en el entorno de programación, empaquetamiento (marshalling)/desempaquetamiento (unmarshalling) y despachar las peticiones al procedimiento adecuado.
2. Gestionar las comunicaciones
3. Enlazado (Binding): Localizar al servidor de un servicio.

La interfaz no es mas que un numero de procedimiento acordado entre cliente y servidor. Este numero viaja dentro del mensaje RPC transmitido por la red. En la parte cliente existe un procedimiento de 'stub' encargado de empaquetar/desempaquetar los argumentos de la llamada, convertir la llamada local en una llamada remota. Esto supone enviar un mensaje, esperar la respuesta y retornar los resultados. En la parte del servidor esta el despachador, junto con el conjunto de procedimientos de stub de servidor, que tienen una misión similar a los de la parte cliente. El despachador selecciona el procedimiento de stub adecuado a partir del numero de procedimiento requerido.

El compilador de IDL genera los procedimientos de stub de cliente y de servidor, las operaciones de empaquetamiento y desempaquetamiento así como los archivos de cabecera necesarios.

Las peticiones de los clientes son con respecto a un nombre de servicio. En ultima instancia deben ser dirigidas a un puerto en el servidor. En un sistema distribuido un binder es un servicio separado que mantiene una tabla que contiene correspondencias de nombres de servicios con puertos de servidor. Se trata, como veremos más adelante, de un servicio de nombres. Importar un servicio es pedir al binder que busque el nombre del interfaz y retorne el puerto del servidor. Exportar un servicio es registrarlo de cara el binder. El binder deberá estar en un puerto bien conocido o, al menos, se le podrá localizar.

Una implementación bastante habitual de RPC es la de SUN. Incorpora todo un conjunto de primitivas para trabajar con RPC en lenguaje C. Dispone de una representación neutral de los datos para su empaquetamiento, XDR (External Data Representation). XDR también se denomina al IDL que se proporciona.

El compilador de IDL, que se denomina rpcgen, genera los procedimientos de stub, el procedimiento main del servidor y el despachador, el código de conversión de los parámetros a XDR, así como los archivos de cabecera correspondientes.

El binder recibe el nombre de port mapper. Cuando se activa un servidor en una máquina remota ésta se registra con el port mapper, obteniendo un puerto de comunicación a través del cuál escuchar. Cuando se quiere acceder a un servicio hay que contactar con el port mapper de la máquina remota, preguntándole por el nombre de un determinado servicio, devolviéndose el puerto de comunicación en el que está a la escucha el servidor correspondiente. Por tanto para acceder a una determinada operación de un servicio hace falta la siguiente información: [host:servicio:procedimiento].

## Servidores Pesados vs Clientes Pesados

[Lewandowsky 1008] realiza una discusión de este concepto dentro del marco de los sistemas cliente-servidor. Los especialistas en sistemas de información califican como 'pesada' (fat) una parte

de un sistema con una cantidad de funcionalidad desproporcionada. Por el contrario, una parte de sistema se considera ligera (thin) si tiene menos responsabilidades [Orfali 1996].

En un sistema cliente-servidor la porción del servidor casi siempre mantiene los datos, mientras que la porción del cliente es responsable de la interfaz de usuario. El desplazamiento de la lógica de la aplicación constituye la distinción entre clientes 'pesados' y servidores 'pesados'. Los sistemas servidores 'pesados' delegan más responsabilidad de la lógica de la aplicación en los servidores, mientras que los clientes 'pesados' dan al cliente mayor responsabilidad. Ejemplo de servidor pesado es un servidor Web, mientras que muchos de los clientes en sistemas de bases de datos constituyen clientes 'pesados'.

Aunque los sistemas basados en servidores 'pesados' han sido los más utilizados en el pasado, en la actualidad muchos diseñadores prefieren sistemas con clientes 'pesados', debido a que son más fáciles de implementar. Los clientes 'pesados' permiten a los usuarios crear aplicaciones y modificar los *front-end* del sistema fácilmente, pero a costa de reducir la encapsulación de los datos; cuanto más responsabilidad se coloque en un cliente, el cliente requerirá un conocimiento más íntimo de la organización de los datos del servidor.

Por otra parte, un servidor 'pesado' es más fácilmente explotable, esto es, más fácil de explotar. Además este tipo de servidor asegura una mayor compatibilidad entre clientes y servidores. Por ejemplo, una página Web diseñada bajo este modelo supondría que no hay disponibilidad de applets de Java, plugins o ActiveX's debido a que el usuario está usando un cliente 'ligero', (un navegador básico), y el servidor estaría restringido al estándar HTML 2.0. El uso de este modelo de cliente 'ligero' asegura que todos los usuarios visualizan una página "aceptable", aunque no se pueden proveer las características avanzadas disponibles con un cliente 'pesado'.

## **Sistemas N-Tiered**

El modelo canónico cliente-servidor asume exactamente dos participantes discretos en el sistema. Se denomina sistema 'two-tier'; la lógica de la aplicación puede estar en el cliente, el servidor, o compartida entre los dos. También puede darse el caso de tener la lógica de la aplicación separada de los datos y de la interfaz de usuario, convirtiéndose el sistema en 'three-tiered'. En un sistema 'three-tiered' ideal toda la lógica de la aplicación reside en una capa separada. Esto ocurre raramente en los sistemas actuales. Siempre hay ciertas partes que permanecen o bien del lado del cliente o del lado del servidor.

Las aplicaciones Web estándar son un ejemplo clásico de sistemas 'three-tiered'. Por un lado se tienen la interfaz de usuario, provista por la interpretación de HTML, por un navegador. Los componentes embebidos visualizados por el navegador residen en la capa media; pueden ser applets de Java, ActiveX's o cualquier otra clase de entidad que provea lógica de aplicación para el sistema. Por último se tienen los datos suministrados por el servidor Web.

## **Ventajas de los sistemas cliente-servidor**

La principal ventaja de los sistemas cliente-servidor está en la correspondencia natural de las aplicaciones en el marco cliente-servidor. Un ejemplo de esto es una agenda electrónica. Debido a que los datos son relativamente estáticos y son visto de manera uniforme por todos los usuarios del sistema parece lógico colocarlos en un servidor que acepte peticiones sobre dichos datos. Es más, en este caso la lógica de aplicación debería estar colocada del lado del servidor, para proporcionar una mayor flexibilidad al sistema de búsquedas (cambios en los algoritmos, etcétera...).

Como resultado de la disponibilidad de middleware compatible para múltiples plataformas y de los



avances recientes de la interoperabilidad binaria, los sistemas cliente-servidor pueden conectar clientes ejecutándose en una plataforma con servidores ejecutándose en otra plataforma completamente distinta. Las tecnologías como Java y los ORBs (Object Request Brokers), de los que trata en profundidad este trabajo, esperan proveer una total integración de todas las plataformas en unos pocos años. Si las porciones de un sistema cliente-servidor encapsulan una única función y siguen un interfaz perfectamente definido, aquellas partes del sistema que proveen los servicios pueden ser intercambiadas sin afectar a otras porciones del sistema. Esto permite a los usuarios, desarrolladores y administradores adecuar el sistema con sus necesidades en cada momento.

Otra ventaja es la posibilidad de ejecutar aplicaciones que hacen uso intensivo de los recursos en plataformas hardware de bajo coste. También el sistema es más escalable, pudiéndose añadir tanto nuevo clientes como nuevos servidores.